Title: CPU Scheduling and Performance Analysis Using SJF (Shortest Job First)

1. AIM

To study and implement the Shortest Job First (SJF) CPU scheduling algorithm using the C programming language and calculate the Completion Time (CT), Waiting Time (WT), Turnaround Time (TAT), Average Waiting Time (AWT), and Average Turnaround Time (ATAT) for a given set of processes.

2. THEORY

2.1 Introduction to CPU Scheduling

CPU scheduling is an important function of an operating system that determines which process will be allocated the CPU for execution.

When multiple processes are present in the ready queue, the operating system uses CPU scheduling algorithms to select the next process for execution.

The main objectives of CPU scheduling are:

1. Maximum CPU utilization.
2. Minimum waiting time.
3. Minimum turnaround time.
4. Maximum throughput.
5. Fair allocation of CPU resources.
6. Efficient utilization of system resources.

2.2 Introduction to SJF Scheduling

Shortest Job First (SJF) is a CPU scheduling algorithm in which the process having the smallest CPU burst time is selected for execution first.

In non-preemptive SJF scheduling, once a process starts execution, it continues until its CPU burst is completed.

The operating system examines the processes available in the ready queue and selects the process with the shortest burst time.

If two processes have the same burst time, the process with the earlier arrival time is selected first.

2.3 Working of SJF Scheduling

The working procedure of SJF scheduling is as follows:

1. All processes enter the ready queue according to their arrival times.
2. The operating system checks the processes that have arrived.
3. The process with the shortest burst time is selected.
4. The selected process executes until completion.
5. After completion, the scheduler checks the remaining available processes.
6. The next process with the shortest burst time is selected.
7. The procedure continues until all processes are completed.

2.4 Types of SJF Scheduling

SJF scheduling is classified into two types.

1. Non-Preemptive SJF

In non-preemptive SJF, once a process gets the CPU, it cannot be interrupted by another process.

The process completes its entire execution before another process is selected.

2. Preemptive SJF (Shortest Remaining Time First)

In preemptive SJF, a running process can be interrupted if another process arrives with a shorter remaining execution time.

The CPU is allocated to the process having the shortest remaining burst time.

2.5 Important Scheduling Terms

1. Arrival Time (AT)

Arrival Time is the time at which a process enters the ready queue and becomes available for execution.

2. Burst Time (BT)

Burst Time is the total amount of CPU time required by a process to complete its execution.

3. Completion Time (CT)

Completion Time is the time at which a process finishes its execution.

4. Turnaround Time (TAT)

Turnaround Time is the total time taken by a process from its arrival until its completion.

Formula:

$$TAT = CT - AT$$

5. Waiting Time (WT)

Waiting Time is the total time a process spends waiting in the ready queue.

Formula:

$$WT = TAT - BT$$

6. Average Waiting Time (AWT)

Average Waiting Time is the average waiting time of all processes.

Formula:

$$AWT = \frac{\sum WT}{n}$$

7. Average Turnaround Time (ATAT)

Average Turnaround Time is the average turnaround time of all processes.

Formula:

$$ATAT = \frac{\sum TAT}{n}$$

Where, n represents the total number of processes.

2.6 Gantt Chart

A Gantt chart is a graphical representation of process execution over time.

It shows the order in which processes are executed, along with their starting and completion times.

Example:

```
None
0        4        7        12

|  P2   |  P3   |  P1   |
```

The Gantt chart helps in understanding the execution sequence of processes.

2.7 Example of SJF Scheduling

Consider the following processes:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Scheduling Order:

- P1 executes first because it is the only available process at time 0.
- At time 7, P2, P3 and P4 are available.
- P3 is selected because it has the shortest burst time.
- P2 is selected next because it arrived before P4 and both have equal burst times.
- P4 executes last.

Gantt Chart:

```
None
0              7    8        12        16

|      P1       |P3  |   P2    |   P4    |
```

Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

Average Waiting Time:

$$AWT = \frac{0+6+3+7}{4}$$

$$AWT = \frac{16}{4} = 4$$

Average Turnaround Time:

$$ATAT = \frac{7+10+4+11}{4}$$

$$ATAT = \frac{32}{4} = 8$$

2.8 Advantages of SJF Scheduling

1. Provides minimum average waiting time under standard non-preemptive SJF assumptions.
2. Short processes receive CPU execution earlier.
3. Improves scheduling efficiency.
4. Reduces waiting time for shorter processes.
5. Suitable for batch-processing environments when burst times can be estimated.

2.9 Disadvantages of SJF Scheduling

1. CPU burst time must be known or estimated in advance.
2. Long processes may experience starvation.
3. It is difficult to predict the exact execution time of processes.
4. Frequent arrival of short processes can delay long processes.
5. It is less suitable for interactive systems when burst times are unpredictable.

2.10 Difference Between FCFS and SJF

FCFS	SJF
First-Come, First-Served	Shortest Job First
Based on arrival time	Based on shortest burst time among available processes
Simple to implement	More complex to implement
May produce higher waiting time	Can minimize average waiting time under standard assumptions
Uses FIFO principle	Uses burst-time selection

Long processes may delay short ones	Long processes may experience starvation

3. CODE

Program: Implementation of Non-Preemptive SJF CPU Scheduling Algorithm in C

Filename: `sjf.c`

```
None
#include <stdio.h>


int main()

{

    int n, i, j;


    int at[20], bt[20];

    int ct[20], tat[20], wt[20];

    int pid[20], completed[20] = {0};


    int current_time = 0;

    int completed_count = 0;

    int shortest;


    float avg_wt = 0;
```

```
float avg_tat = 0;


printf("Enter number of processes: ");
scanf("%d", &n);


if (n <= 0 || n > 20)
{
    printf("Invalid number of processes.\n");
    return 1;
}


// Input process details
for (i = 0; i < n; i++)
{
    pid[i] = i + 1;


    printf("\nEnter Arrival Time of P%d: ", pid[i]);
    scanf("%d", &at[i]);


    printf("Enter Burst Time of P%d: ", pid[i]);
    scanf("%d", &bt[i]);
```

```
    if (at[i] < 0 || bt[i] <= 0)
    {
        printf("Invalid process details.\n");
        return 1;
    }
}

// Non-preemptive SJF scheduling
while (completed_count < n)
{
    shortest = -1;

    // Find shortest available process
    for (i = 0; i < n; i++)
    {
        if (completed[i] == 0 &&
            at[i] <= current_time)
        {
            if (shortest == -1 ||
                bt[i] < bt[shortest] ||
```

```
            (bt[i] == bt[shortest] &&
             at[i] < at[shortest]))
        {
            shortest = i;
        }
    }
}

// If no process has arrived, advance time
if (shortest == -1)
{
    current_time++;
    continue;
}

// Execute selected process
current_time += bt[shortest];

ct[shortest] = current_time;

// Calculate turnaround time
```

```
        tat[shortest] =
            ct[shortest] - at[shortest];

        // Calculate waiting time
        wt[shortest] =
            tat[shortest] - bt[shortest];

        completed[shortest] = 1;

        completed_count++;

        avg_wt += wt[shortest];

        avg_tat += tat[shortest];
    }

    // Calculate averages
    avg_wt = avg_wt / n;
    avg_tat = avg_tat / n;

    // Display results
```

```
    printf("\nSJF Scheduling Results\n");


    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");


    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i],
               ct[i], tat[i], wt[i]);
    }


    printf("\nAverage Waiting Time = %.2f\n",
           avg_wt);


    printf("Average Turnaround Time = %.2f\n",
           avg_tat);


    return 0;
}
```

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.
- Terminal: Linux Terminal.

Step 1: Open Linux Terminal

Press `Ctrl + Alt + T` to open the terminal.

Step 2: Check GCC Compiler

```
None

gcc --version
```

If GCC is not installed, execute:

```
None
sudo apt update

sudo apt install build-essential
```

Step 3: Create a Practical Directory

```
None

mkdir SJF_Scheduling

cd SJF_Scheduling
```

Step 4: Create the C Program

```
None

nano sjf.c
```

Copy and paste the complete C program provided above.

Save the file:

- Press `Ctrl + O`.
- Press Enter.

- Press `Ctrl + X`.

Step 5: Compile the Program

Execute:

```
None
gcc sjf.c -o sjf
```

If there are no compilation errors, the executable file will be generated.

Step 6: Execute the Program

```
None
./sjf
```

Step 7: Enter Sample Input

```
None
Enter number of processes: 4


Enter Arrival Time of P1: 0

Enter Burst Time of P1: 7


Enter Arrival Time of P2: 2

Enter Burst Time of P2: 4


Enter Arrival Time of P3: 4

Enter Burst Time of P3: 1
```

```
Enter Arrival Time of P4: 5

Enter Burst Time of P4: 4
```

Step 8: Observe the Output

Expected Output:

```
None
SJF Scheduling Results


PID     AT      BT      CT      TAT     WT

P1      0       7       7       7       0

P2      2       4       12      10      6

P3      4       1       8       4       3

P4      5       4       16      11      7


Average Waiting Time = 4.00

Average Turnaround Time = 8.00
```

5. RESULT

The Shortest Job First (SJF) CPU scheduling algorithm was successfully implemented using the C programming language.

The program successfully performed the following operations:

1. Accepted arrival time and burst time for multiple processes.
2. Selected the process having the shortest burst time among the available processes.
3. Implemented non-preemptive CPU scheduling.
4. Calculated completion time for each process.

5. Calculated waiting time and turnaround time.
6. Computed average waiting time and average turnaround time.

For the given sample input:

- Average Waiting Time = 4.00 time units.
- Average Turnaround Time = 8.00 time units.

6. CONCLUSION

In this practical, the Shortest Job First (SJF) CPU scheduling algorithm was studied and implemented using C programming. Processes were selected according to their shortest burst time among the processes available in the ready queue. The completion time, waiting time and turnaround time were calculated for each process. Average waiting time and average turnaround time were also determined. Thus, the practical helped in understanding non-preemptive CPU scheduling and performance optimization.